

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE ENERO- JUNIO 2026

Global IA-SA

ACOSTA MARTINEZ MOISES - 22210278

DOCENTE:

ARNULFO ALANIS GARZA

19 DE mayo DE 2026

Introducción

Actualmente, la tecnología y la inteligencia artificial permiten desarrollar herramientas innovadoras para apoyar el aprendizaje y desarrollo personal. En este contexto, SocialMirror es una aplicación móvil diseñada para ayudar a personas neurodivergentes a mejorar sus habilidades sociales mediante simulaciones de interacción y análisis de emociones. Este proyecto integra tecnologías modernas para ofrecer una solución accesible, práctica y orientada al aprendizaje autónomo.

PRIMERA ENTREGA

Decisión de emoción:

Dentro de las funcionalidades de la app, se encuentra la predicción de la emoción que se muestra en la cámara, dicha imagen, dependiendo de que emoción ofrezca será regresada con un mensaje, ya sea que no se entendió, o corroborando la emoción que muestra

```
import torch import torch.nn as nn from torchvision
import transforms, models from PIL import Image
emociones = ["happy", "neutral", "sad"]

# 🌱 Modelo PRO model = models.resnet18(weights=None) model.fc =
nn.Linear(model.fc.in_features, len(emociones))
# Cargar modelo entrenado model.load_state_dict(torch.load("model_resnet.pth",
map_location="cpu")) model.eval()

# Transformaciones transform =
transforms.Compose([
transforms.Resize((224, 224)),
transforms.ToTensor()
]) def generar_feedback(emocion, confianza):
    if confianza < 0.4:
        return "No estoy muy seguro intenta una expresión más clara."
    if emocion == "happy":
        return f"Bien 😊 (confianza {confianza:.2f}), te ves feliz."    elif emocion == "sad":
```

```

        return f"Parece que estás triste 😞 (confianza {confianza:.2f})."    elif emocion ==
"neutral":
        return f"Estás neutral 😊 (confianza {confianza:.2f})."    else:
        return "No se detectó claramente."
def predecir_emocion(imagen_path):    try:
    img = Image.open(imagen_path).convert("RGB")    img =
transform(img)    img = img.unsqueeze(0)
    with torch.no_grad():        output
= model(img)
        prob = torch.softmax(output, dim=1)
confianza, pred = torch.max(prob, 1)
        emocion = emociones[pred.item()]
confianza_valor = confianza.item()
        print("EMOCION:", emocion)
        print("CONFIANZA:", confianza_valor)
        feedback = generar_feedback(emocion, confianza_valor)
        return emocion, feedback
    except Exception as e:        print("ERROR:", e)        return
"neutral", "Error al procesar imagen"

```

Código corriendo:

Tomando el código corriendo como base, se muestran todas las acciones y peticiones que se le hagan dentro de la aplicación como: la emoción, la respuesta del usuario, la respuesta enviada y la confianza

Confianza es lo seguro que es que la emoción sea la correcta, dicho eso, si la emoción es 0.41 o menos seguro, la app arroja un mensaje diciendo que no está seguro y pidiendo que sea otra vez tomada la foto

```
192.168.1.66 - - [11/May/2026 18:10:33] "GET /pregunta HTTP/1.1" 200 -  
  
🔥 NUEVA PETICIÓN 🔥  
💬 Texto: bien  
📁 Imagen guardada en: uploads\1778548262.jpg  
➡ Ejecutando modelo...  
EMOCION: sad  
CONFIANZA: 0.45564594864845276  
🕒 Tiempo modelo: 0.45s  
😊 Emoción detectada: sad  
✅ RESPUESTA ENVIADA  
  
192.168.1.66 - - [11/May/2026 18:11:02] "POST /emocion HTTP/1.1" 200 -  
192.168.1.66 - - [11/May/2026 18:11:02] "POST /siguiente HTTP/1.1" 200 -  
192.168.1.66 - - [11/May/2026 18:11:03] "GET /pregunta HTTP/1.1" 200 -  
  
🔥 NUEVA PETICIÓN 🔥  
💬 Texto: bien  
📁 Imagen guardada en: uploads\1778548283.jpg  
➡ Ejecutando modelo...  
EMOCION: neutral  
CONFIANZA: 0.5692247152328491  
🕒 Tiempo modelo: 0.48s  
😊 Emoción detectada: neutral  
✅ RESPUESTA ENVIADA  
  
192.168.1.66 - - [11/May/2026 18:11:24] "POST /emocion HTTP/1.1" 200 -  
192.168.1.66 - - [11/May/2026 18:11:24] "POST /siguiente HTTP/1.1" 200 -  
192.168.1.66 - - [11/May/2026 18:11:24] "GET /pregunta HTTP/1.1" 200 -
```



La aplicación iniciaría con una pantalla de inicio con su logo y dos botones.

El botón que dice iniciar simulación, envía a la segunda pestaña, la cual permitiría observar todo tranquilamente

El botón con el símbolo de un micrófono, permite iniciar la toma de imagen automáticamente y hablando con la ia, principalmente para las personas que ya tengan experiencia usándola



Dentro de la segunda pestana, observamos el mensaje que la app nos envíe, iniciando con la comunicación, al responderle. Automáticamente nos mostrara la cámara, así pudiendo tomar la foto.

La forma de responder puede ser escribiendo o hablando, cualquiera de los dos los mostrara en la caja de texto



Terminando con la funcionalidad y tomando la foto, la aplicación nos arrojará: su respuesta, la pregunta que te hizo, la respuesta que se le dio y la emoción que emoción tenemos

Para seguir únicamente tendríamos que responder a su comentario de la misma manera.

En caso de que la emoción de una foto no corresponda a lo que dijo, la app lo detectará y hará mención a eso

Ajustes requeridos de primera entrega

En la primera entrega se identificaron diversos problemas en el proceso de entrenamiento del modelo. En primer lugar, se observó que el modelo no lograba aprender correctamente, ya que el número de epochs era insuficiente. Por ello, se planteó la necesidad de aumentar las iteraciones de entrenamiento para permitir que el modelo analizara los datos en más ocasiones y pudiera mejorar su desempeño.

Además, no solo existía una falta de aprendizaje, sino que en varios casos el modelo aprendía de manera incorrecta, generando predicciones poco precisas. Esto indicaba la necesidad de ajustar tanto el proceso de entrenamiento como los datos de entrada.

Otro aspecto importante fue el preprocesamiento de las imágenes. Se detectó que las imágenes no estaban normalizadas, lo cual afectaba el rendimiento del modelo. Por esta razón, se consideró necesario aplicar una normalización adecuada, asegurando que todas las imágenes tuvieran el mismo formato, tamaño y características, con el fin de mejorar la consistencia y facilitar el aprendizaje del modelo.

Conclusión

El desarrollo de la aplicación SocialMirror permitió demostrar el potencial de la inteligencia artificial como herramienta de apoyo en el aprendizaje de habilidades sociales para personas neurodivergentes. A través de la integración de tecnologías como el reconocimiento de emociones, el análisis de voz y la simulación de interacciones, se logró crear una plataforma funcional, accesible y orientada a las necesidades del usuario.

Este proyecto no solo cumple con los objetivos planteados inicialmente, sino que también abre la posibilidad de futuras mejoras y ampliaciones, como la incorporación de más escenarios interactivos, mayor precisión en los modelos de inteligencia artificial y personalización avanzada de la experiencia del usuario.

En conclusión, SocialMirror representa una propuesta innovadora que contribuye al uso responsable y socialmente significativo de la tecnología, demostrando que las aplicaciones móviles pueden ser una herramienta clave para promover la inclusión, el desarrollo personal y el aprendizaje continuo.

SEGUNDA ENTREGA

1. Dataset utilizado

Se emplea el dataset **FER2013**, el cual contiene imágenes de rostros en escala de grises con dimensiones de 48x48 píxeles, organizadas en distintas categorías emocionales.

Las imágenes se dividen en:

- Conjunto de entrenamiento (train)
- Conjunto de prueba (test)

2. Preprocesamiento de datos

Antes del entrenamiento, las imágenes pasan por un proceso de transformación:

- Conversión a escala de grises
- Redimensionamiento a 48x48 píxeles
- Aplicación de aumentos de datos:
 - Rotación aleatoria
 - Volteo horizontal
- Conversión a tensor

Este preprocesamiento permite mejorar la capacidad de generalización del modelo.

3. Arquitectura del modelo

Se implementa una **Red Neuronal Convolutiva (CNN)** compuesta por:

Capas convolucionales

Encargadas de extraer características de las imágenes:

- 3 capas convolucionales
- Activación ReLU
- Normalización por lotes (BatchNorm)
- Reducción de dimensión mediante MaxPooling

- Aplanamiento de características
- Capa densa intermedia (256 neuronas)

- Dropout (0.5) para evitar sobreajuste
- Capa de salida con 7 clases (emociones)

4. Proceso de entrenamiento

El entrenamiento se realiza mediante un ciclo iterativo controlado por el número de **epochs**.

Un *epoch* representa una iteración completa sobre todo el conjunto de entrenamiento.

En este caso:

- Se utilizan 30 epochs
- El modelo aprende progresivamente en cada iteración

Los datos se dividen en grupos llamados *batches*:

- Tamaño de batch: 32 imágenes

Esto permite:

- Optimizar el uso de memoria
- Acelerar el entrenamiento
- Mejorar la estabilidad del aprendizaje

El modelo recibe las imágenes y genera una predicción:

- Entrada: imágenes
- Salida: probabilidades de cada emoción

Se utiliza:

- **CrossEntropyLoss**

Esta función mide la diferencia entre la predicción del modelo y la etiqueta real.

El objetivo es minimizar esta pérdida.

5. Problemas comunes en el entrenamiento

El modelo memoriza los datos en lugar de generalizar.

Solución:

- Uso de Dropout
- Aumento de datos

El modelo no aprende lo suficiente.

Solución:

- Incrementar epochs
- Mejorar la arquitectura

Algunas emociones aparecen más que otras.

Efecto:

- El modelo tiende a predecir clases más frecuentes (ej. neutral)

Tabla comparativa, primera y segunda entrega

Aspecto	Segunda entrega	Primera entrega
Propósito	Entrenar un modelo de reconocimiento de emociones	Crear y guardar la estructura del modelo sin entrenar
Dataset	Utiliza FER2013 (train y test)	No utiliza dataset
Preprocesamiento	Grayscale, resize, rotación y flip	No incluye transformaciones
DataLoader	Sí, usa batches	No utiliza
Arquitectura	CNN profunda (3 capas + BatchNorm + Dropout)	CNN simple (2 capas)
Número de clases	4 emociones	3 emociones
Entrenamiento	Sí (con epochs)	No
Epochs	30	No aplica
Función de pérdida	CrossEntropyLoss	No aplica
Evaluación	Calcula precisión	No evalúa
Uso de GPU/CPU	Automático (CUDA si disponible)	No configura dispositivo
Precisión	Calcula accuracy	No mide rendimiento
Guardado del modelo	Modelo entrenado	Modelo vacío
Complejidad	Alta	Baja

Imágenes normalizadas agregadas



Archivo anterior crear_modelo.py

```
import torch
import torch.nn as nn

# Clases
emociones = ["angry", "fear", "happy", "sad", "surprise", "neutral"]

class EmotionModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 32, 3),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, 3),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 10 * 10, 128),
            nn.ReLU(),
            nn.Linear(128, len(emociones))
        )

    def forward(self, x):
        x = self.conv(x)
        x = self.fc(x)
        return x

# Crear modelo
model = EmotionModel()

# Guardar modelo vacío (estructura correcta)
torch.save(model.state_dict(), "model.pth")

print("✓ Modelo creado como model.pth")
```

Modificacion a archivo crear_modelo.py

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# Rutas
train_dir = r"C:\Users\moise\OneDrive\Escritorio\fer2013\train"
test_dir = r"C:\Users\moise\OneDrive\Escritorio\fer2013\test"

# Transformaciones
transform = transforms.Compose([
    transforms.Grayscale(),
    transforms.Resize((48,48)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor()
])

def main():
    print("📁 Cargando dataset...")

    # 📁 Dataset
    train_data = datasets.ImageFolder(train_dir, transform=transform)
    test_data = datasets.ImageFolder(test_dir, transform=transform)

    print(f"✅ Imágenes de entrenamiento: {len(train_data)}")
    print(f"✅ Imágenes de prueba: {len(test_data)}")

    # DataLoader
    train_loader = DataLoader(train_data, batch_size=32, shuffle=True,
num_workers=0)
    test_loader = DataLoader(test_data, batch_size=32, num_workers=0)

    # Modelo CNN
    class EmotionModel(nn.Module):
        def __init__(self):
            super(EmotionModel, self).__init__()

            self.conv = nn.Sequential(
                nn.Conv2d(1, 32, 3, padding=1),
                nn.ReLU(),
                nn.BatchNorm2d(32),
```

```

        nn.MaxPool2d(2),

        nn.Conv2d(32, 64, 3, padding=1),
        nn.ReLU(),
        nn.BatchNorm2d(64),
        nn.MaxPool2d(2),

        nn.Conv2d(64, 128, 3, padding=1),
        nn.ReLU(),
        nn.BatchNorm2d(128),
        nn.MaxPool2d(2)
    )

    self.fc = nn.Sequential(
        nn.Flatten(),
        nn.Linear(128 * 6 * 6, 256),
        nn.ReLU(),
        nn.Dropout(0.5),
        nn.Linear(256, 7)
    )

    def forward(self, x):
        x = self.conv(x)
        x = self.fc(x)
        return x

# Configuración
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"📱 Usando dispositivo: {device}")

model = EmotionModel().to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.0005)

# ENTRENAMIENTO
epochs = 30

print("🚀 Iniciando entrenamiento...")

for epoch in range(epochs):
    model.train()
    total_loss = 0

```

```

    for batch_idx, (images, labels) in enumerate(train_loader):
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()

        total_loss += loss.item()

        if batch_idx % 100 == 0:
            print(f"Epoch {epoch+1} | Batch {batch_idx} | Loss: {loss.item():.4f}")

            print(f"🔥 Epoch {epoch+1} completado | Loss total: {total_loss:.2f}")

# EVALUACIÓN
print("🔍 Evaluando modelo...")

model.eval()
correct = 0
total = 0

with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)

        outputs = model(images)
        _, predicted = torch.max(outputs, 1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

accuracy = 100 * correct / total
print(f"\n🎯 Precisión en test: {accuracy:.2f}%")

# GUARDAR MODELO
torch.save(model.state_dict(), "model.pth")
print("💾 Modelo guardado como model.pth")

```



```
if __name__ == "__main__":  
    main()
```

Conclusión

El entrenamiento del modelo consiste en un proceso iterativo en el cual la red neuronal aprende a identificar patrones en imágenes mediante la minimización de una función de pérdida. A través del uso de múltiples epochs, optimización con Adam y evaluación con métricas de precisión, se logra desarrollar un sistema capaz de clasificar emociones faciales.